Information Fusion 138 (2027) 104698



Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus



Full Length Article

MVRFDet: Malicious package detection in PyPI ecosystem using multi-view representation fusion

Jiale Yan, Bo Zhao*

School of Cyber Science and Engineering, Wuhan University, Wuhan, 430072, Hubei, China

ARTICLE INFO

Keywords:
Malicious package detection
Python package index (PyPI)
Multi-view representation fusion
Large language model
Graph neural network
Software supply chain security

ABSTRACT

The widespread use of the Python Package Index has made it a primary target for malicious package poisoning attacks, seriously threatening software supply chain security. However, existing detection methods often rely on single-view features, making it difficult to cope with increasingly complex and covert attack methods, and it is difficult to achieve an effective balance between detection performance and efficiency. To address these issues, this paper proposes a novel malicious package detection method called MVRFDet, based on multi-view representation fusion. Specifically, this method fuses features from three views: package metadata, global API call graph of source code and fine-grained control flow graph, and code semantic features of suspicious API code context snippets. To reduce runtime overhead, a centrality analysis and large language model are used to automatically extract a list of sensitive APIs offline, accurately locating suspicious API code context snippets. Finally, a channel attention module is designed to adaptively fuse representations from three views to achieve accurate classification. Extensive experiments on mixed datasets from multiple sources demonstrate that MVRFDet achieves state-of-the-art performance compared to eight baseline methods, with improvements in precision ranging from 2.5% to 24.8% and recall from 4% to 18.3%. Ablation studies and parameter sensitivity analyses validate the effectiveness of the model's component design and the optimal selection of hyperparameters. Furthermore, detection efficiency evaluations show that it achieves a better trade-off between runtime overhead and detection performance.

1. Introduction

With the widespread adoption of the Python programming language and the rapid development of its ecosystem, the Python Package Index (PyPI) has become a crucial platform for global developers to share and access software components [1]. However, as a mainstream open-source software repository, PyPI's openness and ease of use have attracted numerous attackers who upload malicious software packages disguised as legitimate tools, posing a serious security threat to downstream users and projects. Therefore, accurately and efficiently detecting malicious software packages within the PyPI ecosystem has become a critical issue that urgently needs to be addressed [2].

Malicious package detection within the PyPI ecosystem is essential for maintaining the security of the software supply chain. First, once a malicious package successfully launches an attack, it can cause significant economic losses and sensitive data leaks to users and organizations, with an extremely wide-ranging impact. Second, the increasingly sophisticated and covert attack methods employed by attackers make it difficult for existing detection methods to effectively identify new types of malicious behavior, increasing the difficulty of defense [3].

Currently, some related work has been dedicated to solving the problem of malicious package detection, but many shortcomings remain [4]. Existing detection methods mainly employ two technical approaches: static analysis and dynamic analysis. Static analysis methods are broadly categorized into signature-based methods [5], rule-based methods [6,7], metadata-based methods [8], traditional machine learning-based methods [9–11], and large language model (LLM)-based methods [12–14]. Dynamic analysis methods [15,16] monitor the behavior of packages during actual installation and execution in an isolated environment to determine their malicious nature. This approach effectively combats code obfuscation and camouflage, directly capturing the true malicious behavior of the package.

The main bottleneck of static analysis lies in the fact that while signature-based, rule-based, and metadata-based methods are efficient, they lack runtime behavior insights and generate a large number of false positives [17]. Dynamic analysis methods, while achieving good detection results, require significant resources and time, failing to meet the needs of large-scale real-time detection in package registry centers [4]. Traditional machine learning methods require manual feature selection for model training, and their effectiveness heavily depends on

* Corresponding author.
E-mail addresses: jialeyan@whu.edu.cn (J. Yan), zhaobo@whu.edu.cn (B. Zhao).

https://doi.org/10.1016/j.inffus.2026.104698
Received 8 January 2026; Received in revised form 7 August 2026; Accepted 8 August 2026
Available online 9 August 2026
1566-2535/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

feature quality [18]. The method of detection combined with LLM shows a good prospect, still face the challenge of long detection times [12–14]. The above analysis highlights a key challenge: how to accurately detect malicious packages with acceptable time constraints? Recent risk assessment research has identified unpatched software, weak access controls, human errors, and limited security expertise as important factors contributing to system vulnerabilities [19]. These findings further motivate the development of automated and scalable mechanisms for detecting potentially malicious software packages before they affect downstream systems. Looking back at current detection methods, most employ single-view features, which often have limitations and are prone to false positives or false negatives. How to realize multi-view representation of software packages and construct a multi-level and comprehensive representation fusion detection system is an inevitable trend in addressing increasingly complex and covert malicious package poisoning attacks.

Based on the above analysis and insights, to address the aforementioned challenges, we propose a novel MVRFDet method. MVRFDet leverages multi-view representation fusion and the powerful analytical capabilities of offline LLM to overcome the limitations of existing methods, achieving accurate and efficient detection of malicious packages. Specifically, firstly, MVRFDet extracts the metadata features of the package. Then, it extracts the API call graph from the source code of the package through static analysis, encoding it to obtain global coarse-grained graph structure features. Using a pre-extracted list of sensitive APIs automatically generated by LLM, it locates suspicious APIs in the call graph, mapping them to obtain suspicious API code context snippets from the source code. Next, it constructs a control flow graph and encodes the code semantic information, thereby extracting fine-grained graph structure features and code semantic features. Finally, attention is used to fuse features from three views: metadata, graph structure, and code semantics, to achieve accurate malicious package classification.

To evaluate the effectiveness of MVRFDet, extensive experiments were conducted on a mixed dataset collected from Guo et al. [20], MalwareBench [21], OSCAR [15], and the official PyPI dataset [22]. Experimental results show that MVRFDet achieves the best performance across all performance metrics compared to eight baseline methods. Specifically, this method improved precision by 2.5%–24.8% and recall by 4%–18.3%. Ablation studies and parameter sensitivity analyses validated the effectiveness of the model's component design and the selection of optimal hyperparameters. Furthermore, the evaluation results of detection efficiency show that MVRFDet achieves a better trade-off between runtime overhead and detection performance.

The contribution of this work can be summarized as:

- We propose MVRFDet, a novel PyPI malicious package detection method based on multi-view representation fusion. This method utilizes the fusion of package metadata, graph, and code representations to construct a detector.
- We propose a method combining centrality analysis with LLM to achieve offline automatic extraction of a sensitive API list, which can automatically update the sensitive API list as malicious behavior evolves.
- We design a channel attention module to adaptively assign fusion weights to representations from different views.
- We conduct extensive experiments to evaluate and verify the effectiveness of MVRFDet.

The structure of this paper is as follows: Section 2 provides background and related work. Section 3 introduces the research motivation and the proposed MVRFDet method. Section 4 presents the evaluation and results, including the experimental environment, implementation details, baseline methods, experimental results and discussion. Section 5 discusses and describes the limitations. Section 6 summarizes the entire paper and outlines future work.

2. Background and related work

2.1. Security threats in OSS supply chain

To systematically understand the complex threat landscape of the open-source software (OSS) supply chain, researchers have conducted in-depth explorations from two perspectives: the entire lifecycle and key links.

At the macro-classification level, the pioneering work of Ladisa et al. [3] constructed an attack classification method covering the entire lifecycle of open-source software package from introduction, development, construction to distribution and operation, providing a clear theoretical framework and analytical perspective for this field.

In terms of threat analysis at key links, research focuses on the core nodes of the supply chain. First, the software package ecosystem and source code repositories have become the main attack frontiers. The large-scale empirical research by Gu et al. [2] reveals the prevalence of threats such as malicious package injection in the mainstream package registry. Meanwhile, analyses of source code repositories have also developed automated techniques for detecting malicious submissions [23,24] and malware hidden in project forks [25].

Second, automated pipelines constitute new risk exposures. Systematic assessments of continuous integration and continuous deployment (CI/CD) services [26] and GitHub actions workflows [27] have exposed serious vulnerabilities such as key leakage and build artifact contamination. To ensure the integrity of the build process, reproducible build techniques [28] have been proposed to prevent compile-time tampering, while research on compiler instability [29] further highlights the challenge of ensuring build determinism.

Finally, the high dependence of modern software on third-party libraries makes dependency networks the core of risk propagation. Researchers have used software component analysis techniques [30,31] to deeply characterize the path and impact of vulnerabilities propagating across ecosystems in complex dependency trees [32]. However, even if the impact of vulnerabilities is clear, timely remediation still faces huge obstacles [33]. The research of Enck and Williams [34] clearly points out that updating vulnerability dependencies is the core challenge facing the industry, and compatibility risks and resource constraints are key factors hindering rapid response. In summary, existing research collectively depicts a multi-dimensional threat landscape of the OSS supply chain and points out that building a defense-in-depth system covering the entire process is an essential requirement for ensuring its security.

2.2. Malicious package detection

The most serious area of open-source software supply chain attacks is the poisoning of malicious packages in open-source package registries (such as PyPI and NPM). Malicious attackers use these platforms to spread malicious packages. Such attacks are highly covert and have a wide impact, attracting widespread attention from academia and industry [3]. Therefore, how to efficiently and accurately detect malicious packages has become a core issue in ensuring supply chain security [2]. This section will systematically review existing malicious package detection efforts from different technical dimensions.

Before delving into detection techniques, understanding the common methods used by attackers is crucial. Package obfuscation is a typical method of malicious package poisoning. Attackers create packages with names similar to popular packages (such as typosquatting) to trick developers into downloading them. The research of Neupane et al. [35] goes beyond the traditional typosquatting scope, providing a more in-depth analysis of package obfuscation attacks, systematically classifying and demonstrating various obfuscation techniques, and providing an important attacker perspective and knowledge base for subsequent targeted detection work.

Static analysis methods analyze package without executing the code, and are currently the mainstream technology for detecting

malicious packages. Static analysis methods can be broadly categorized into signature-based methods, rule-based methods, traditional machine learning-based methods and LLM-based methods.

Signature-based and rule-based methods rely on known malicious features. Signature-based methods, such as VirusTotal [5], identify threats by comparing the package's code with a database of known malicious packages. Their advantage is high accuracy against known threats, but they cannot detect zero-day attacks. Rule-based methods, such as Microsoft's OSS Detect Backdoor [6] and Bandit4Mal [7] for Python, identify potential malicious behaviors through predefined rule sets, such as detecting suspicious API calls or Base64 encoded payloads. These methods are highly interpretable, but the rule base has high maintenance costs and is easily bypassed by advanced obfuscation techniques.

To overcome the limitations of rule-based methods, researchers have turned to data-driven traditional machine learning methods. These methods typically extract features from code or metadata to train classification models. For example, Amalfi [9] proposed a practical automated detection framework for the NPM ecosystem. SAP [10] explored the feasibility of malicious package detection across the NPM and PyPI ecosystems. MPHunter [11] innovatively uses unsupervised code clustering technology to identify malicious packages as outliers in normal packages, achieving good results in the PyPI ecosystem. Metadata-based methods bypass code analysis and directly use the package's metadata information for judgment, such as publisher information and package description text. Research by Halder et al. [8] shows that metadata features alone can effectively distinguish between malicious and benign packages. This method has low computational overhead and can quickly filter out suspicious packages, but it may not be able to detect carefully crafted attacks at the code level.

With the breakthroughs in code understanding and generation by LLMs, they have also been applied to malicious package detection. LLM can deeply understand the semantics and context of code, thereby identifying malicious logic that has been complexly obfuscated. EA4MP [12] uses the BERT [36] model to learn deep code behavior features and then combines them with metadata features to improve the ability to detect malicious packages in PyPI. Cerebro [13] constructed a unified BERT model, which can detect malicious packages in NPM and PyPI simultaneously by analyzing abstract malicious behavior sequences. Zahan et al. [14] also directly explored the potential of using LLM to detect malicious packages in NPM, demonstrating the great advantage of this technical route in understanding complex attack patterns.

In contrast to static analysis, dynamic analysis methods monitor the behavior of packages by actually executing the installation and operation process in an isolated sandbox environment, such as file system operations, network connections, system calls, etc., to determine whether they are malicious. This method can effectively resist code obfuscation and camouflage, and directly capture the real malicious behavior of packages. Zheng et al. [15] proposed a robust dynamic detection framework for industrial environments. This framework combines fuzzing and aspect-based behavior monitoring to achieve good detection results in NPM and PyPI. Huang et al. proposed DONAPI [16], which improves the accuracy and interpretability of detection by constructing a behavior sequence knowledge mapping to perform refined analysis of the dynamic behavior of NPM packages.

In summary, malicious package detection has evolved from signature-based and rule-based approaches relying on prior knowledge to data-driven models that integrate static code, metadata, and runtime behavior. However, malicious package detection is not a stationary classification problem. Attackers continuously adjust their strategies to evade existing defenses, resulting in changes in package metadata, code structures, API usage patterns, obfuscation techniques, and payload-triggering mechanisms. Such adversarial evolution may cause concept drift, making the discriminative evidence learned from historical packages less reliable for newly emerging threats.

Existing detection approaches also exhibit view-specific limitations. Static analysis is efficient and scalable but may generate false positives and lose effectiveness against obfuscation, dynamic loading, and staged payload delivery. Dynamic analysis can directly capture runtime behavior but incurs substantial execution overhead and may miss conditionally triggered malicious logic. Metadata-based methods are lightweight but can be evaded through carefully crafted package profiles. More importantly, the reliability of these views varies across attack strategies and package instances. A fixed fusion mechanism may overemphasize a compromised or weakly informative view, limiting its robustness in an evolving threat environment.

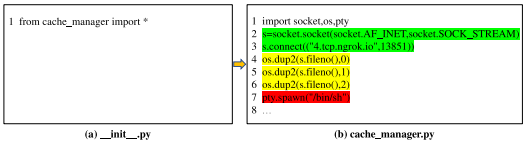


Fig. 1. In the *seccache-0.0.3* malicious package, malicious script *cache_manager.py* is called by the *import* function in *_init_.py*.

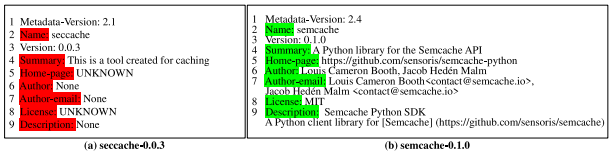


Fig. 2. Comparison of metadata information between malicious package *seccache-0.0.3* and benign package *semcache-0.1.0* with similar names.

To address these limitations, we propose MVRFDet, a multi-view representation fusion approach for malicious package detection. Instead of assigning fixed importance to heterogeneous evidence, MVRFDet adaptively models the relative contributions of complementary views for each package. This design enables the detector to emphasize informative evidence while reducing its dependence on potentially unreliable views, thereby improving detection robustness and efficiency under diverse and evolving attack patterns.

3. Methodology

In this section, we first introduce our research motivation. Then, we describe in detail MVRFDet, a novel and effective method for malicious package detection in the PyPI ecosystem.

3.1. Research motivation

In this section, we explain our research motivation for performing multi-view representation of software packages. We analyzed the recently disclosed PyPI malicious package *seccache-0.0.3*, whose malicious code is shown in Fig. 1. The malicious package *seccache-0.0.3* imports all modules from the *cache_manager.py* file in the *_init_.py* file, which contain malicious behavior. As shown in Fig. 1(b), the attacker imports various modules in line 1. In lines 2–3, a network connection is established to the remote server *4.tcp.ngrok.io:13851*. In lines 4–6, the standard stream is redirected, redirecting the process's standard input, output, and error to the socket. This means that all keyboard input and screen output will be transmitted over the network. Finally, in line 7, a pseudo-terminal is created and the /bin/sh shell is started. This shell inherits the redirected input and output, forming complete remote control.

Different from the attack methods of previous malicious packages, the malicious package *seccache-0.0.3* no longer places the malicious code trigger in *setup.py*, meaning it is not triggered when the user installs the malicious package. Instead, the attacker places the malicious code trigger in the *_init_.py* file. The malicious link is only accessed or sensitive information is sent back when the user imports the package, which provides these malicious packages with extremely strong concealment. Our

detection and analysis revealed many malicious packages with similar attack methods.

As shown in Fig. 2, we compare and display the metadata information of the malicious package *seccache-0.0.3* and the benign package *semcache-0.1.0* with a similar name. We observed systematic differences in their metadata features. Regarding the completeness of the package metadata information, the malicious package *seccache-0.0.3* lacks key fields, such as the *Home-page* field, the *Author* field, the *Author-Email* field, the *License* field, and the *Description* field, while the benign package *semcache-0.1.0* provides complete relevant information. In terms of the quality of package functionality description, the description field of the malicious package is None, while the benign package provides detailed technical explanations. In summary, attackers primarily focus on embedding malicious functionality within packages, rather than ensuring the overall completeness of package design. Furthermore, due to practical constraints such as development effort and attack cost, malicious package maintainers often fail to achieve perfect camouflage in package metadata.

Based on the above analysis of the code and metadata of the malicious package *semcache-0.1.0*, we obtained the following observations: (1) **Malicious behavior originates from certain sensitive API operations.** The sensitive API call graph sequence in Fig. 1(b), such as *socket.socket()*, *s.connect()*, *os.dup2()*, and *pty.spawn()*, represents the relevant attack patterns. However, graph-based features often rely on topological structures and lack in-depth understanding of code intent, making it difficult to distinguish between normal network communication and malicious data transmission. Therefore, it is necessary to deeply understand the context of the code corresponding to sensitive APIs and extract code features. (2) **Metadata-based information provides effective malicious clues beyond the code.** These malicious clues are blind spots that are easily overlooked in static analysis. Because they are relatively easy to forge, they cannot be used as a sole basis for judgment.

In summary, graph-based features provide the skeleton of attack logic and reveal the structural topology of the code. Code-based features fill in the flesh and blood of attack details, enabling semantic understanding of the specific payload and intent of the attack. Metadata-based features serve as identity fingerprints, indirectly verifying the credibility of the package. This study aims to overcome the limitations and vulnerabilities of a single structural perspective by fusing features from multiple views, thereby enhancing the ability to generalize to attack evolution and achieving accurate identification of similar highly concealed malicious packages.

3.2. Overview of MVRFDet method

As shown in Fig. 3, MVRFDet mainly consists of five steps: 1) Metadata Feature Extraction, 2) Code Feature Extraction, 3) Graph Feature Extraction, 4) Feature Matrix Generation, and 5) Channel Attention Fusion Classification.

- **Metadata feature extraction:** Given a PyPI package, the PKG-INFO file is first read, and predefined fields are extracted. These are then embedded using a Metadata Encoder to obtain the metadata view feature.
- **Code feature extraction and graph feature extraction:** Given a PyPI package, static analysis tools are first used to extract the APIs call graph from the source code. Then, a list of sensitive APIs pre-obtained automatically by an LLM is used to perform sensitive APIs localization on the API call graph, mapping it back to the source code to obtain suspicious APIs code context snippets. Next, static analysis is used to construct the control flow graph of the suspicious APIs code context snippets. The obtained API call graph and control flow graph are encoded using a graph encoder to obtain the graph view feature. The obtained code context snippets text is encoded using a code encoder to obtain the code view feature.

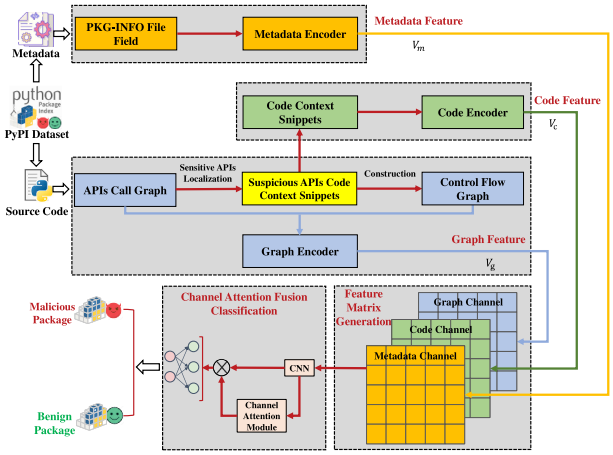


Fig. 3. Overall framework diagram of the MVRFDet method. The proposed method contains five key aspects: 1) **Metadata Feature Extraction**, 2) **Code Feature Extraction**, 3) **Graph Feature Extraction**, 4) **Feature Matrix Generation**, and 5) **Channel Attention Fusion Classification**.

- **Feature matrix generation:** After encoding three features from different perspectives using the PyPI package, these three perspective features are treated as three channels and reorganized to generate a three-channel feature matrix.
- **Channel attention fusion classification:** Inspired by the concept of (R, G, B) channels in images, the three-channel feature matrix is used as input, and channel attention fusion feature is employed to classify malicious packages. Next, we describe each key aspect of the proposed method in detail.

3.3. Metadata feature extraction

Based on the discussion in Section 3.1 Research motivation and the inspiration from previous work on malicious package detection using only metadata information, we chose to use information from the perspective of package metadata as an important feature for detecting malicious packages. Specifically, we selected seven fields and their attribute values from the PKG-INFO file of the PyPI package as input to the metadata encoder. As shown in Fig. 2, the seven selected fields are as follows: Name, Summary, Home-page, Author, Author-Email, License, and Description. We selected the pre-trained model *all-mpnet-base-v2* of the advanced text embedding tool sentence-transformers [37] as the metadata encoder to encode the metadata, obtaining a 768-dimensional metadata feature vector V_m.

3.4. Code feature extraction and graph feature extraction

To obtain features from different perspectives and at different granularities in the PyPI package, we first use the static analysis tool Scalpel [38] to extract the API call graph from the package's source code. This graph abstracts the high-level semantic call logic of the APIs and encodes it using a graph encoder, representing coarse-grained global perspective information at the package level. Since the API call graph is typically very large, encoding it using a graph encoder may not be able to identify and capture fine-grained local perspective information within the package. To address this challenge, we first locate the suspicious APIs in the API call graph using a list of sensitive APIs, and further map the source code to extract suspicious APIs code context snippets. Then, we use Scalpel to construct a control flow graph for the code context snippets and use a graph encoder to further extract fine-grained local perspective features. Finally, to further capture the semantics of the code context snippets, we use a code encoder to extract fine-grained local perspective features related to sensitive code and suspicious behavior.

3.4.1. Automatic offline extraction of sensitive APIs list

Overview. Malicious software packages often exhibit malicious behavior across multiple functions and files, primarily executing their malicious activities by calling relevant APIs. Previous research [13,15,39,40] manually collected lists of sensitive APIs related to malicious logic using observation and induction. This approach is time-consuming, tedious, laborious, and unable to dynamically adapt to the evolution of malicious packages. To address this issue, we employ centrality analysis combined with LLM to automate the extraction of the sensitive APIs list. It is important to note that the sensitive APIs list extraction is performed before the feature extraction in the MVRFDet method. A single extraction can be reused across multiple package detection tasks. To adapt to the evolution of malicious packages, the sensitive API list is maintained offline through periodic full updates and event-driven incremental updates. A full update can be performed regularly or before a new detector release, while an incremental update is triggered when newly verified malicious packages, previously unseen attack patterns, or false-negative cases are identified. Newly emerging APIs are re-evaluated using the same centrality analysis and dual LLM verification pipeline. Each updated list is versioned and validated before deployment, without introducing additional online detection overhead.

To achieve automated extraction of the sensitive APIs list, we draw inspiration from graph centrality analysis methods in the field of Android malware detection [41], which has proven effective in identifying frequently called sensitive APIs. Centrality analysis is commonly used in social network analysis, where centrality metrics measure the importance or influence of nodes within the network structure. Structurally, API call graphs and social networks share modeling similarities in form and characteristics. For example, an API call graph can be viewed as a social network, API nodes as different member nodes in the social network, and API call relationships as social interactions among members. Existing research [4,41] has shown that sensitive APIs of malware typically have higher and more anomalous centrality metrics compared to benign software in API call graphs.

Calculate centrality measures. We first used the static analysis tool Scalpel to generate an API call graph for each of all collected malicious packages (see Table 1), and then calculated the centrality values of all API nodes. To ensure that no API node has a centrality value of 0, we incremented all API centrality values by 1 by default. We selected six widely used centrality metrics to ensure that the comprehensive analysis yielded the most accurate list of sensitive APIs. These six centrality metrics are: Degree centrality, Closeness centrality, Katz centrality, Harmonic centrality, Eigenvector centrality, and Betweenness centrality, which are detailed as follows. The above centrality measures can be obtained via NetworkX [42].

Degree centrality. Degree centrality measures the immediate influence of a node based on its number of direct connections within the graph. A higher degree indicates greater direct connectivity and potential influence.

$$V_D = \frac{deg(a)}{(N-1)} \tag{1}$$

where $deg(a)$ is the number of edges incident to node a, and N is the total number of nodes.

Closeness centrality. Closeness centrality measures the average distance from a node to all other nodes in a graph. The higher the closeness centrality of a node, the shorter the average path from it to other nodes.

$$V_C = \frac{(N-1)}{\sum_{a \neq b} d(a,b)} \tag{2}$$

where $d(a,b)$ is the shortest path length between nodes a and b.

Harmonic centrality. Harmonic centrality is a variant of closeness centrality that sums the reciprocals of shortest path lengths. It is robust in disconnected graph since unreachable nodes contribute zero.

$$V_H = \sum_{a \neq b} \frac{(N-1)}{d(a,b)} \tag{3}$$

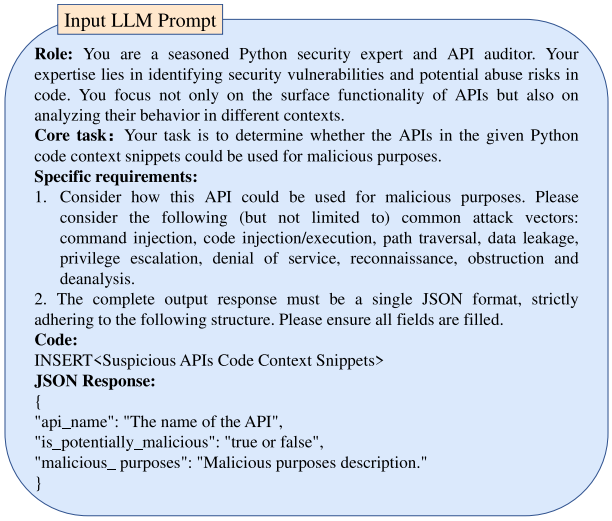


Fig. 4. Role-based prompt engineering of sensitive API verification.

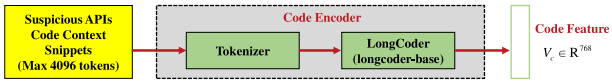


Fig. 5. A detailed description of the code encoder used in the MVRFDet method to encode suspicious APIs code context snippets.

If a is unreachable from b, the term is treated as zero.

Katz centrality. Katz centrality measures the influence of a node, taking into account the contribution of all paths to that node, but with a smaller weight for longer paths (achieved through a decay factor α). Furthermore, it assigns a base centrality β to each node to ensure that the centrality of all nodes is non-zero.

$$V_K = \alpha \sum_b A_{ab} V_b + \beta \tag{4}$$

where A is the adjacency matrix, $\lambda_{\max}$ is the largest eigenvalue of A, α is the attenuation factor ($0 < \alpha < 1/\lambda_{\max}$), and β is the base centrality (often set to 1).

Eigenvector centrality. Eigenvector centrality assigns relative importance to a node based on the importance of its neighbors. A node is important if it is connected to other important nodes.

$$V_E = c \sum_b A_{ab} V_b \tag{5}$$

where A is the adjacency matrix, c is a proportionality constant, and $A_{ab} = 1$ when a is connected with b, otherwise $A_{ab} = 0$.

Betweenness centrality. Betweenness centrality quantifies the number of times a node acts as a bridge along the shortest path between two other nodes.

$$V_B = \sum_{x \neq a \neq y} \frac{\delta_{xy}(a)}{\delta_{xy}} \tag{6}$$

where δ_{xy} is the total number of shortest paths from node x to node y, and $\delta_{xy}(a)$ is the number of paths that pass through node a.

Sensitive APIs list extraction based on centrality measure ranking and LLM evaluation. To identify sensitive APIs, we summed the centrality values of all APIs with the same name for each type, then divided the sum by the total number of malicious packages to obtain the average centrality value for each API. We then sorted the APIs based on their average centrality values and selected the top N as a list of candidate sensitive APIs.

To improve the accuracy of identifying sensitive APIs, we attempted to leverage the powerful code understanding capabilities of LLM to evaluate the N candidate APIs. Drawing on the experience of manually collecting sensitive APIs in previous research [13,15,39,40], to minimize

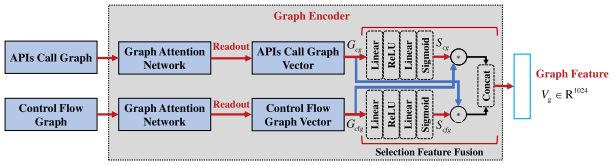


Fig. 6. A detailed description of the graph encoder used in the MVRFDet method to encode APIs call graph and control flow graph.

omissions, we increased the number of candidate APIs compared to manual selection, selecting 500 as candidate sensitive APIs. Finally, to minimize the impact of irrelevant APIs, we analyzed and evaluated the suspicious APIs code context snippets corresponding to all candidate sensitive APIs using role-based prompt engineering with LLM (DeepSeek-R1 [43] and GPT-5 [44]) to determine their potential for malicious use. After analysis and evaluation by two LLMs, a detailed and comprehensive list of sensitive APIs was compiled, containing 293 sensitive APIs. For details on the specific prompt engineering, please refer to Fig. 4.

3.4.2. Suspicious APIs code context snippets

We locate suspicious APIs in the APIs call graph using the sensitive APIs list, and further map the source code to extract suspicious APIs code context snippets. Specifically, for each identified suspicious API, we select up to 4096 tokens as code context snippets centered on its location, thereby obtaining fine-grained local perspective information on suspicious packages. It should be noted that the maximum length of the code context snippets is 4096 tokens, primarily because 4096 is the maximum encoding length currently supported by the code encoder. If the context window exceeds the file boundary, it is clipped to the valid token range, and the remaining token budget is allocated to the other side whenever possible. When multiple suspicious APIs occur within the same file, their context snippets are independently extracted and encoded, even if the corresponding windows partially overlap. This design preserves the local semantic context of each suspicious API. Completely identical snippets are deduplicated before encoding to avoid unnecessary computation. Finally, the snippet-level representations are aggregated using mean pooling to obtain the package-level code feature vector V_c.

3.4.3. Code encoder

We chose the advanced code pretraining model LongCoder [45] released by Microsoft. LongCoder is a sparse and efficient pre-trained Transformer model designed for modeling long code, primarily employing a sliding window mechanism to achieve self-attention computation. LongCoder supports encoding long code with up to 4096 tokens, making it ideal for modeling fine-grained local code features of our extracted suspicious API code context snippets. Specifically, we first load the *longcoder-base* model from Huggingface to obtain its Tokenizer, then perform tokenization on the code context snippets. Next, we load the *longcoder-base* model to encode the code context snippets, obtaining a 768-dimensional code feature vector V_c. The workflow of the code encoder is shown in Fig. 5.

3.4.4. Graph encoder

To obtain graph-view features from the API call graph and control flow graph in the PyPI package source code, inspired by previous work [46], we designed a graph encoder as shown in Fig. 6. We first employ a multi-layer Graph Attention Network (GAT) [47] to encode the two types of graphs. Then, we aggregate and update the features of the graph nodes. Next, we use a readout function to aggregate all node features to obtain a graph-level representation. Finally, we use a selection feature fusion mechanism to fuse the two types of graph features into the final graph feature V_g.

When aggregating information, GAT can assign different weights to embeddings of different nodes within the same neighborhood, enabling more refined feature updates and thus better performance on graph data. Specifically, the workflow of the GAT layer can be summarized as follows: First, the input graph node features are projected onto a new feature space through a linear transformation. Then, attention scores between nodes are computed, and normalization is performed using softmax function. Finally, attention weights are used to aggregate neighbor information to obtain new node representations. The formal description is as follows.

The initial node features of the GAT layer input are $\boldsymbol{H} = \{h_1, h_2, \ldots, h_N\}, h_i \in \mathbb{R}^d$, where N is the number of graph nodes and d is the feature dimension.

First, the original features are projected into a new feature space using a shared weight matrix W, as shown in Eq. (7) below:

$$h_i' = W \cdot h_i \tag{7}$$

where $h_i \in \mathbb{R}^d$ is the original feature vector of node i, $W \in \mathbb{R}^{d' \times d}$ is the learnable shared weight matrix, and $h_i' \in \mathbb{R}^{d'}$ is the transformed feature vector.

Next, attention scores between nodes are calculated. These scores reflect the importance of one node to another during information aggregation. For edge e_{ij}, the attention score is calculated as follows:

$$e_{ij} = \text{LeakyReLU}\left(a^\top [h_i' \parallel h_j']\right) \tag{8}$$

where h_i' and $h_j' \in \mathbb{R}^{d'}$ are the feature vectors of node i and node j after transformation, $[h_i' \parallel h_j']$ is the concatenation of the feature vectors of the two nodes, a is the learnable attention vector, and LeakyReLU is the non-linear activation function. The calculated attention scores have inconsistent ranges, so the softmax function is needed to normalize the scores of all neighbors for each node, as shown in the following formula:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \tag{9}$$

where α_{ij} is the normalized attention weight, i.e. the importance of node i's neighbor j, and $\mathcal{N}(i)$ is the set of neighbor nodes of node i.

Finally, attention weights are used to aggregate the features of neighbors to update the representation of itself, as shown in the following formula:

$$h_i'' = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \cdot h_j'\right) \tag{10}$$

where h_i'' is the updated representation vector of node i, α_{ij} is the attention weight of node i to node j, h_j' is the transformed feature vector of node j, and σ is the ELU activation function.

As increasing GNN depth may introduce representation degradation, optimization difficulty, and generalization issues [48], we employ a relatively shallow stack of four GAT layers to balance representation capacity and training stability. Residual connections are not used in the current implementation. Incorporating skip connections to support deeper GAT architectures remains a potential direction for future optimization. After learning the node representations through these 4 layers, we aggregate all node representations in the graph into a graph-level representation vector G by applying mean-pooling, as shown in the following formula:

$$G = \frac{h_1^{\text{final}} \oplus \ldots \oplus h_N^{\text{final}}}{N} \tag{11}$$

where h_i^{final} is the final feature vector of node i, $\oplus$ is the element-wise addition operation, and N is the number of nodes in the graph. For disconnected CFGs, GAT performs message passing within each connected component, and all node representations are aggregated through mean pooling. For a degenerate single-node CFG, we retain the node and add a self-loop. If no valid CFG node is generated, a zero vector is used as the representation to ensure stable model training. For convenience, we use G_{cg} and G_{cfg} to represent the graph representation vectors extracted from the APIs call graph and control flow graph, respectively.

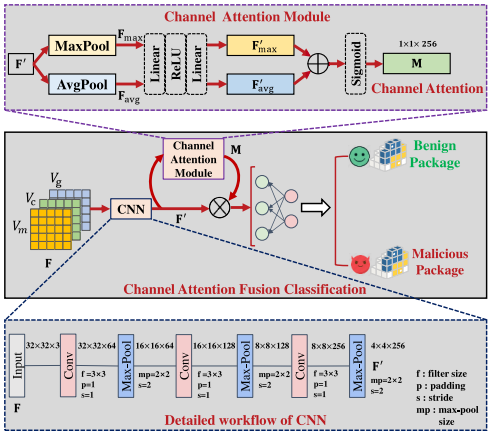


Fig. 7. A detailed description of the channel attention module and CNN used in the MVRFDet method to fuse multi-view representation for classification.

To better utilize the extracted global coarse-grained feature G_{cg} from the API call graph and the local fine-grained feature G_{cfg} from the control flow graph, we design a selection feature fusion mechanism to fuse G_{cg} and G_{cfg} into the final graph feature Vg, as shown in Fig. 6. We employ two selectors, each containing two linear layers, using one ReLU activation function, and finally using the sigmoid function to obtain the selection vector. First, the graph vectors G_{cg} and G_{cfg} are passed through two selectors that do not share parameters, where the sigmoid function scales each element of the vector to [0,1]. We use the scaled vectors as selection vectors S_{cg} and S_{cfg} to cross-select the corresponding graph vectors G_{cg} and G_{cfg}, ultimately obtaining the graph feature vector V_g. This selection mechanism helps the model filter out unimportant information. The selection feature fusion mechanism can be formalized as follows:

$$S_{cg} = \mathrm{Sigmoid}\big(W_{12}\mathrm{ReLU}\big(W_{11}G_{cg} + b_{11}\big) + b_{12}\big) \tag{12}$$

$$S_{cfg} = \mathrm{Sigmoid}\big(W_{22}\mathrm{ReLU}\big(W_{21}G_{cfg} + b_{21}\big) + b_{22}\big) \tag{13}$$

$$V_g = \mathrm{Concat}\big(S_{cg} \otimes G_{cfg}, S_{cfg} \otimes G_{cg}\big) \tag{14}$$

where $W11, W12, W21, W22$ and $b11, b12, b21, b22$ are the weights and biases of the linear layer, respectively. The $\otimes$ is the element-wise multiplication operation. Different from the commonly used self-weighting strategy, we adopt a cross-selection mechanism to further exploit the complementarity between the two graph representations. The cross-selection operation in Eq. (14) is designed to capture the conditional relevance between the two graph representations at different granularities, thereby enhancing their complementarity. The API call graph representation G_{cg} captures package-level, coarse-grained API calling patterns, whereas the control flow graph representation G_{cfg} characterizes fine-grained execution logic surrounding suspicious API calls. Instead of independently weighting each representation using its own selector, we use the selector derived from one graph view to filter the other graph view. In this way, global calling patterns provide contextual guidance for selecting informative local control-flow features, while local execution logic helps identify the most relevant dimensions of the global API call representation.

3.5. Feature matrix generation and channel attention fusion classification

We extract representations from three complementary views of each PyPI package, including the metadata feature vector V_m, the code feature vector V_c, and the graph feature vector V_g. Specifically, the metadata encoder and the code encoder generate 768-dimensional representation $V_m \in \mathbb{R}^{768}$ and $V_c \in \mathbb{R}^{768}$, respectively. For the graph view, the API call graph and control flow graph are separately encoded into 512-dimensional graph-level representations. After selective feature fusion and concatenation according to Eq. (14), the graph encoder produces a 1024-dimensional representation $V_g \in \mathbb{R}^{1024}$. Inspired by successful three-channel feature extraction in current image processing, we treat the three view features as three channels. Because the three feature vectors have different dimensions and originate from heterogeneous encoders, directly stacking them may introduce dimensional inconsistencies and scale differences. We therefore perform lightweight multi-view feature alignment before constructing the feature matrix. First, the metadata and code feature vectors are padded with zero elements from 768 to 1024 dimensions, while the graph feature vector remains unchanged, as shown in the following formula.

$$\begin{aligned} V_m^{pad} &= \mathrm{Pad}(V_m, 1024) \\ V_c^{pad} &= \mathrm{Pad}(V_c, 1024) \\ V_g^{pad} &= V_g \end{aligned} \tag{15}$$

Next, view-wise L2 normalization is applied independently to the three padded vectors.

$$\hat{V}_i = \frac{V_i^{pad}}{\left\| V_i^{pad} \right\|_2 + \epsilon}, \quad i \in \{m, c, g\} \tag{16}$$

where ϵ is a small constant used to avoid division by zero. This operation ensures that the three aligned representations have comparable numerical scales while preserving the zero-valued padding elements. Finally, the normalized vectors $\hat{V}_m$, $\hat{V}_c$, and $\hat{V}_g$ are reshaped into three 32×32 matrices and stacked along the channel dimension, as shown in the middle of Fig. 7.

$$\mathrm{F} = \mathrm{Stack}\left(\mathrm{R}(\hat{V}_m), \mathrm{R}(\hat{V}_c), \mathrm{R}(\hat{V}_g)\right) \in \mathbb{R}^{3\times32\times32} \tag{17}$$

In the field of image processing, numerous studies have demonstrated that convolutional neural network (CNN) can effectively learn representations from multi-channel inputs. Therefore, we employ a CNN model as a feature extractor for a three-channel feature matrix. To adaptively fuse the features extracted by the CNN, we designed a channel attention module, as shown in Fig. 7.

The detailed workflow of the CNN model is shown in the lower part of Fig. 7. The input feature matrix $\mathbf{F}$ is processed through three convolutional layers and three max-pooling layers to extract features $\mathbf{F}'$. The specific parameters for the convolution and max-pooling operations are shown in Fig. 7.

To adaptively fuse features from three perspectives, we designed a channel attention module that adaptively assigns weights to representations from different perspectives, enabling MVRF to focus on information relevant to malicious intent, as shown in the upper part of Fig. 7. First, max-pooling and average-pooling operations are performed on feature $\mathbf{F}'$ to obtain features $\mathbf{F}_{\max}$ and $\mathbf{F}_{avg}$. Then, the two features are passed through a shared multi-layer perceptron (MLP) to learn features $\mathbf{F}'_{\max}$ and $\mathbf{F}'_{avg}$. Finally, feature $\mathbf{F}'_{\max}$ and $\mathbf{F}'_{avg}$ are element-wise added, and then passed through a sigmoid function to obtain the channel attention vector $\mathbf{M}$. The formal description of channel attention calculation is shown in the following formula:

$$\mathbf{M} = \sigma\big(\mathrm{MLP}\big(\mathrm{MaxPool}(\mathbf{F}')\big) \oplus \mathrm{MLP}\big(\mathrm{AvgPool}(\mathbf{F}')\big)\big) \tag{18}$$

where $\oplus$ is an element-wise addition operation. The two linear layers of the MLP contain 16 and 256 neurons, respectively.

In the classification stage, we fuse the features $\mathbf{F}'$ extracted by the CNN and the channel attention vector $\mathbf{M}$ learned by the channel attention module, and use MLP to obtain the final classification result $\hat{y}$ in the PyPI package, which is formally described as follows:

$$\hat{y} = \mathrm{Softmax}\big(\mathrm{MLP}\big(\mathbf{F}' \otimes \mathbf{M}\big)\big) \tag{19}$$

where $\otimes$ represents element-wise multiplication operation. The MLP has two linear layers containing 4096 and 2 neurons respectively, with the ReLU activation function applied after the first linear layer.

Table 1
Details of the mixed datasets obtained by collecting PyPI packages from different sources.

Dataset	Malicious	Benign
Guo dataset[20]	9550	\
MalwareBench [21]	140	\
OSCAR [15]	500	\
PyPI official download [22]	\	10,000
Total	10,190	10,000

4. Evaluation and results

In this section, we first introduce the experimental setup and then discuss the experimental results in detail. To comprehensively evaluate the MVRFDet method, we propose and address the following research questions:

- **RQ1 Effectiveness evaluation:** How effective is MVRFDet in detecting PyPI malicious packages compared to other state-of-the-art baseline methods?
- **RQ2 Ablation study:** How does the design of different components affect the effectiveness of MVRFDet?
- **RQ3 Efficiency evaluation:** How is the time cost of MVRFDet?
- **RQ4 Hyperparameter sensitivity analysis:** How sensitive are different hyperparameters to MVRFDet?

4.1. Experimental settings

4.1.1. Implementation details

All experiments in this study were conducted on a dedicated server to ensure the reliability and reproducibility of our results. The server was equipped with an Ubuntu 20.04.6 LTS operating system. The hardware configuration consisted of an Intel(R) Xeon(R) Silver 4314 CPU @ 2.40GHz processor with 16 cores, 64GB of RAM, and an NVIDIA GeForce RTX 4090 GPU with 24GB of video memory. The software environment was built upon Python 3.11.7. For deep learning model development and acceleration, we utilized the PyTorch framework version 1.11.0, in conjunction with CUDA 11.3 and the cuDNN 8.2 library.

The specific implementation details are as follows: In the metadata feature extraction stage, the pre-trained model *all-mpnet-base-v2* of sentence-transformers [37] is used as metadata encoder. In the code feature extraction stage, the pre-trained model *longcoder-base* of code LLM LongCoder [45] is used as the code encoder. In the graph feature extraction stage, a 4-layer GAT [47] and a selection feature fusion module are used as graph encoder. The API call graph and control flow graph used in the graph feature extraction process are extracted using the static analysis tool Scalpel [38]. In the channel attention fusion classification stage, a CNN with 3 convolutional layers, a channel attention module, and a 2-layer MLP are used to classify malicious packages. In the offline automatic extraction phase of the sensitive APIs list, the Scalpel tool is first used to generate an API call graph of all malicious packages in the dataset. Then, the average of the six centrality values of all API nodes with the same name is calculated and sorted. Finally, LLM DeepSeek-R1 [43] and GPT-5 [44] are used to perform role-based prompt engineering analysis and evaluation on the top-500 candidate sensitive APIs obtained from the sorting, and the final sensitive API list containing 293 sensitive APIs is obtained. During the model training, the cross entropy loss function is used, the Adam optimizer is used, the learning rate is set to $2e^{-5}$, the batch size is set to 8, and the number of training epochs is set to 100.

It is important to note that the sensitive API list is treated as a reusable domain-level security knowledge base rather than a trainable component of the downstream classifier. In the primary experimental setting, we construct this list offline from the datasets of verified all malicious packages before downstream model evaluation and keep it fixed across all cross-validation folds. The extraction process uses centrality analysis and LLM-based verification only to identify APIs with potential malicious usage patterns. It does not use benign packages, downstream prediction results, or test-set performance to optimize the classifier. This design is consistent with the intended deployment scenario, in which a sensitive API knowledge base is periodically maintained using confirmed malicious samples and subsequently reused to analyze newly submitted packages.

Table 2
Performance evaluation results for MVRFDet compared to eight state-of-the-art baseline methods. The selected baselines represent different types of methods: VirusTotal (a signature-based online detection method), OSS Detect Backdoor and Bandit4Mal (rule-based methods), Amalfi, SAP, and MPHunter (machine learning-based methods), and EA4MP and Cerebro (LLM methods based on BERT).

Approach	Precision	Recall	F1-score
VirusTotal [5]	88.3	78.5	83.1
OSS Detect Backdoor [6]	73.6	81.9	77.5
Bandit4Mal [7]	79.4	90.5	84.6
Amalfi [9]	83.2	85.7	84.4
SAP [10]	85.3	82.4	83.8
MPHunter [11]	93.6	91.0	92.3
EA4MP [12]	95.9	92.8	94.3
Cerebro [13]	94.3	90.3	92.3
MVRFDet	**98.4**	**96.8**	**97.6**

4.1.2. Datasets

To comprehensively evaluate our proposed MVRFDet method, we constructed a large-scale and balanced datasets. Malicious packages were primarily collected from three sources: the Guo dataset [20], MalwareBench [21], and OSCAR [15], to ensure sample quality and diversity. The malicious packages from the above-mentioned papers have been detected and verified for their maliciousness using various tools and methods. We only performed filtering and deduplication. Benign packages followed the approach of previous research [11,12], collecting 10,000 packages from the PyPI official repository [22] and rigorously screening them using all baseline methods in this paper and manual assistance to ensure their harmlessness. The final datasets contains 10,190 malicious packages and 10,000 benign packages, as shown in Table 1. In the experimental setup, we divided the datasets into training and test sets at a ratio of 80% (8152 malicious packages and 8000 benign packages) and 20% (2038 malicious packages and 2000 benign packages), respectively, maintaining the original distribution of malicious and benign categories. Furthermore, we used five-fold cross-validation to evaluate the model's stability and ensure the reliability of the experimental results.

4.1.3. Evaluation metrics

To conduct a rigorous evaluation of our proposed MVRFDet method for PyPI malicious package detection, we employ a suite of quantitative metrics commonly used in prior research [12,13,15,16]. These indicators are:

1) Precision: The ratio of true malicious packages to all packages predicted as malicious, indicating the model's control over false positives.

$$Precision = \frac{TP}{TP + FP} \tag{20}$$

2) Recall: The ratio of correctly identified malicious packages to the total number of actual malicious packages, reflecting the model's ability to avoid false negatives.

$$Recall = \frac{TP}{TP + FN} \tag{21}$$

3) F1-Score: The harmonic mean of Precision and Recall, which provides a balanced performance measure, especially important for datasets

with skewed class distributions.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{22}$$

4.2. Effectiveness evaluation (RQ1)

Baseline method. For RQ1, we compare MVRFDet with eight state-of-the-art baseline methods. Specifically, we selected a signature-based online detection method VirusTotal [5]; two rule-based methods, OSS Detect Backdoor [6] and Bandit4Mal [7]; three machine learning-based methods, Amalfi [9], SAP [10], and MPHunter [11]; and two BERT-based large model methods, EA4MP [12] and Cerebro [13]. For VirusTotal, OSS Detect Backdoor, and Bandit4Mal, we use the optimal threshold for 3 alerts observed by Zhang et al. [13]. For fair comparison, we use the same hyperparameter settings as in the original paper for SAP and MPHunter. For Amalfi, EA4MP, and Cerebro, we use the optimal model and hyperparameter settings from the original paper. For Amalfi, we train the Amalfi detection model using the optimal decision tree model. For EA4MP, we use an ensemble training of the optimal naive bayes model and the BERT model. For Cerebro, we train the Cerebro detection model using the optimal RoBERTa [49] model.

Setup. We conducted our effectiveness evaluation experiments using all the mixed datasets collected in Table 1. The experiments employed five-fold cross-validation, dividing the datasets into five equal parts for five rounds of training and testing. In each round, four parts were used for training and one part for testing. The average performance metric was then calculated. To ensure fair comparison, we used the same training and testing data for MVRFDet, Amalfi, SAP, MPHunter, EA4MP, and Cerebro. Since the signature-based method VirusTotal and the rule-based methods OSS Detect Backdoor and Bandit4Mal do not involve model training, their performance was evaluated using only the test data in each round.

Results and discussion. The experimental results are shown in Table 2. Our MVRFDet method achieved the best performance across all evaluation metrics compared to eight state-of-the-art baseline methods. Specifically, in terms of precision, MVRFDet achieved the best performance of 98.4%, a 2.5% improvement over the best baseline method, EA4MP (95.9%). In terms of recall, MVRFDet ranked first with a score of 96.8%, a significant 18.3% improvement over the weakest method, VirusTotal (78.5%). Among the baseline methods, LLM-based methods showed strong competitiveness, with EA4MP and Cerebro achieving F1-score of 94.3% and 92.3%, respectively. Rule-based OSS Detect Backdoor and Bandit4Mal, and signature-based VirusTotal, showed significant performance gaps compared to LLM-based methods, with their F1-score generally below 85%. These results demonstrate that MVRFDet possesses excellent and balanced performance on the PyPI malicious package detection task.

The performance bottleneck of baseline methods primarily stems from their inherent limitations: VirusTotal relies on known malicious package signatures, thus limiting its ability to detect zero-day attacks and polymorphic malicious package, explaining its relatively low recall (78.5%). While rule-based methods achieve acceptable recall, their low precision indicates overly broad rules, leading to numerous false positives. For example, OSS Detect Backdoor focuses excessively on backdoor threats, neglecting other covert attacks such as data theft and malicious script downloads, resulting in low precision. Furthermore, rule-based methods cannot effectively model cross-file collaborative attacks and malicious interactions with external resources, severely limiting their detection capabilities.

Machine learning-based methods typically rely on a single feature view, making them vulnerable to detecting complex, multi-stage attacks. For instance, the MPHunter method uses only the *setup.py* script to detect malicious packages, ignoring features from other views and the evolutionary trends of cross-file collaborative attacks. This design flaw can lead to many false negatives.

Table 3
Performance comparison results of different variant approachs.

Approach	Precision	Recall	F1-score
MFC-only	88.1	70.5	78.3
CFC-only	85.4	91.2	88.2
GFC-only	83.7	87.6	85.6
MFC + CFC	93.2	90.8	92.0
MFC + GFC	91.5	89.1	90.3
CFC + GFC	94.3	92.5	93.4
TFC + CNN w/o CAM	96.2	95.1	95.6
MVRFDet	**98.4**	**96.8**	**97.6**

LLM-based methods EA4MP and Cerebro demonstrate significant performance potential, proving the powerful capabilities of LLM in analyzing code semantics. However, due to the inherent illusion problem of LLM, it is prone to false negatives of malicious packages. For example, Cerebro and EA4MP have recall of 90.3% and 92.8%, respectively, significantly lower than MVRFDet's 96.8%, further highlighting its inherent flaws.

MVRFDet's superior performance is mainly attributed to its multi-view feature fusion strategy. By collaboratively analyzing the package's metadata features, graph features, and code features, it fully mines and fuses the deep complementary information between these three views and features of different granularities. This cross-dimensional information fusion enables MVRFDet to effectively reduce false positives (high precision) and false negatives (high recall), thus achieving optimal detection performance in the complex and ever-changing PyPI package.

4.3. Ablation study (RQ2)

For RQ2, we conducted ablation studies to validate the effectiveness of each component. Specifically, we compared the performance of MVRFDet with the following variants.

The effect of different view features. To explore the impact of different view features in our method MVRFDet, we conducted the following ablation study.

Setup. MFC-only (Metadata feature channel-only): MFC-only indicates classification using only the metadata channel in the feature matrix.

CFC-only (Code feature channel-only): CFC-only indicates classification using only the code channel in the feature matrix.

GFC-only (Graph feature channel-only): GFC-only indicates classification using only the graph channel in the feature matrix.

MFC + CFC (Metadata feature channel and code feature channel): MFC + CFC indicates classification using both the metadata channel and code channel in the feature matrix.

MFC + GFC (Metadata feature channel and graph feature channel): MFC + GFC indicates classification using both the metadata channel and graph channel in the feature matrix.

CFC + GFC (Code feature channel and graph feature channel): CFC + GFC indicates classification using both the code channel and graph channel in the feature matrix.

TFC + CNN w/o CAM (Three feature channels using a CNN without channel attention module): TFC + CNN w/o CAM means that three feature channels in the feature matrix are used for classification by the CNN without a channel attention module.

Results and discussion. The results in Table 3 clearly reveal the contribution and synergistic effect of each view feature in MVRFDet. Specifically, in the single-channel feature variant, CFC-only and GFC-only demonstrated strong independent detection capabilities, with F1-score of 88.2% and 85.6%, respectively. In contrast, MFC-only performed the weakest, with an F1-score of only 78.3%, mainly limited by its low recall (70.5%), indicating that relying solely on package metadata information would result in a large number of false negatives. In the dual-channel feature combination, CFC + GFC performed the best,

with an F1-score of 93.4%, indicating a strong complementarity between code content and graph structure information. Other combinations, such as MFC + CFC and MFC + GFC, also achieved F1-score of 92.0% and 90.3%, respectively, significantly outperforming any single-channel feature variant. Finally, the complete model MVRFDet achieved the best performance across all metrics, with its 97.6% F1 -score establishing its superior overall performance. When MVRFDet removed the Channel Attention Module (CAM), the performance of TFC + CNN w/o CAM declined significantly, with the F1-score dropping to 95.6%, demonstrating the crucial role of CAM in integrating multi-channel information.

The ablation experiments not only validated the rationale behind the MVRFDet design but also revealed the core value of multi-view feature fusion and the channel attention mechanism. Experimental results show that any single view has inherent limitations, proving the complementarity and necessity of multi-view feature fusion. The low recall of MFC-only indicates that many malicious packages may be well-disguised at the metadata level. Conversely, while CFC-only and GFC-only can detect threats at the code and graph structure levels, they cannot identify malicious packages with obvious metadata features. Combining the three-view features significantly improved performance, demonstrating the high complementarity of metadata information, code content, and graph structure information. They construct a more comprehensive profile of malicious behavior, effectively making up for the blind spots of single-view features.

Although TFC + CNN w/o CAM possesses information from all views, its equal treatment of all channels may lead to noise interference from non-critical channels in the classification. CAM in MVRFDet can adaptively learn and assign different weights to each channel of different samples. For example, when detecting cross-script API collaborative attacks, CAM automatically assigns higher weights to GFC. When analyzing code injection attacks, CFC and MFC may be more important. This adaptive feature focusing mechanism makes the model more accurate and robust, demonstrating the effectiveness of CAM's adaptive weighting of features from different views.

Each perspective feature has different independent value. Code features are the cornerstone of detection, and their high recall indicates that they are the core of capturing malicious behavior. Graph features provide graph-structured context, which is crucial for understanding complex attack patterns. While metadata features have limited independent performance, they can quickly filter out a large number of low-level threats, providing a more comprehensive view for the model. Their value is maximized after fusion. In summary, ablation experiments strongly demonstrate that the success of MVRFDet stems from the effective integration and intelligent focusing of heterogeneous information from multiple views. This multi-view fusion design philosophy enables it to detect increasingly complex and diverse malicious packages.

The effect of different components of graph encoder. To explore the impact of different components of the graph encoder in MVRFDet, we conducted the following ablation study.

Setup. API CG-only (API call graph-only): API CG-only means that the graph encoder uses only the Graph Attention Network to encode the API call graph.

CFG-only (Control flow graph-only): CFG-only means that the graph encoder uses only the Graph Attention Network to encode the control flow graph.

API CG + CFG w/o SFF (API call graph and control flow graph without selection feature fusion): API CG + CFG w/o SFF means that the graph encoder uses the Graph Attention Network to encode both the API call graph and the control flow graph, without a selection feature fusion module, and only performs a simple concatenation operation.

GE w/ GCN (Graph encoder replaces Graph Attention Network with Graph Convolutional Network): GE w/ GCN means that the graph encoder uses the Graph Convolutional Network [50] to replace the Graph Attention Network to encode both the API call graph and the control flow graph, and includes a selection feature fusion module.

GE w/ GGNN (Graph encoder replace Graph Attention Network with Gated Graph Sequence Neural Network): GE w/ GGNN means that a Gated Graph Sequence Neural Network [51] is used in the graph encoder to replace the Graph Attention Network for encoding the API call graph and control flow graph, and it includes a selection feature fusion module.

Results and discussion. Table 4 shows the experimental results, demonstrating the rationality of the graph encoder design. Among the variants of the graph encoder, the variants GE w/ GCN and GE w/ GGNN, using GCN and GGNN as alternatives, achieved F1-score of 95.7% and 94.3%, respectively. While these performances are respectable, they are still inferior to the GAT-based MVRFDet. When the graph encoder uses the GAT structure and removes the selective feature fusion (SFF) module (API CG + CFG w/o SFF), the performance declines more significantly, with the F1-score dropping to 95.1%. Finally, the variants API CG-only and CFG-only, using only a single graph type, performed the weakest, with F1-score below 91%, highlighting the necessity of fusing information from multiple graph structures.

Experimental results show that using a single-type graph CFG-only in the graph encoder (90.3%) outperforms API CG-only (87.4%) in F1-score performance, but both are limited. This reveals the limitations of single-type graphs. API call graphs abstract high-level semantic call logic of APIs, representing global coarse-grained information at the package level, but are insensitive to the context of specific API calls. Control flow graphs, on the other hand, excel at describing the logical structure of code, representing local fine-grained information of suspicious API code context snippets within a package, but may overlook the dangers of high-level semantic calls of APIs. Therefore, combining the two types of graphs is key to building comprehensive code understanding. The performance improvement of API CG + CFG w/o SFF compared to using a single-type graph structure demonstrates the complementarity of the dual-graph structure.

Simply concatenating features in the graph encoder treats API CG and CFG features equally, which is suboptimal in many cases. For example, when detecting an attack that primarily relies on dangerous API calls, API CG features should dominate. However, when analyzing a package that performs malicious behavior through complex control flow, CFG features are more critical. The SFF module, through a learning mechanism, dynamically assigns different weights to API CG and CFG features based on the context of the input graph, thereby improving precision and recall by 2.8% and 2.1%, respectively.

Comparative experiments with representative graph neural networks GCN and GGNN validate the correctness of choosing GAT as the core of the graph encoder. GCN updates node representations by averaging neighbor nodes, a form of undifferentiated information fusion. While GGNN is powerful, its recurrent neural network-based design is more inclined to capture sequential dependencies in the graph. However, code graphs are inherently complex data structures, and their importance lies more in local neighborhood relationships. The attention mechanism introduced by GAT can learn the importance weights of different neighbor nodes, thus achieving better results than GCN and GGNN. In summary, MVRFDet's graph encoder is not a simple stacking of components, but rather a collaborative effort of complementary dual-type graph structures, intelligent fusion strategies, and advanced encoding architecture to construct graph features with a profound and accurate understanding of the code graph. This is key to achieving advanced performance in malicious package detection tasks.

4.4. Efficiency evaluation (RQ3)

To explore the runtime overhead of MVRFDet in detecting malicious packages, we conducted experiments on the test data in the datasets shown in Table 1. We tested the runtime overhead of the main steps of the MVRFDet method, including metadata feature extraction, graph feature extraction, code feature extraction, graph encoder embedding, and feature matrix generation (FMG) and classification. The left side of

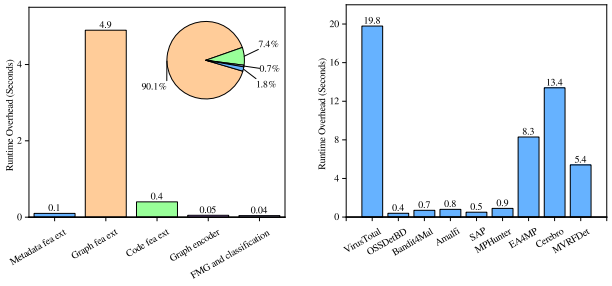


Fig. 8. Comparison results of average running time overhead. Left: Average runtime results of each key step. Right: Average runtime overhead against baseline methods.

Fig. 8 shows the average runtime overhead for each major step. In the metadata feature extraction stage, given the package's metadata information, the metadata encoder is used to encode the metadata feature. The average runtime of this process is 0.1 s. In the graph feature extraction stage, given the package's source code, the API call graph is first obtained. The sensitive APIs list is used to locate the suspicious APIs in the API call graph, and the source code is further mapped to extract suspicious APIs code context snippets. Then, a control flow graph is constructed based on the code context snippets. Finally, the graph encoder is used to encode the graph feature. The average runtime of this process is 4.9 s. In the code feature extraction stage, the code context snippets obtained in the graph feature extraction stage are encoded using the code encoder to obtain the code feature. The average runtime of this process is 0.4 s. Finally, the three features are reshaped into a three-channel feature matrix for classification. The average runtime for this process is 0.04 s. In summary, the total runtime of the MVRFDet method is approximately 5.4 s. Analysis of the results shows that the most significant time overhead is in the graph feature extraction stage, with the graph encoder embedding taking an average of 0.05 s. Therefore, static analysis tools parsing the source code to generate API call graphs and control flow graphs are relatively time-consuming operations. This indicates that future optimization efforts should focus on this stage.

To investigate the detection efficiency of the MVRFDet method compared to baseline methods, we conducted experiments on the test data in Table 1. To ensure a fair comparison, all runtime experiments were conducted under the same hardware environment described in Section 4.1.1. The reported runtime denotes the average per-package detection time after model loading and initialization. Therefore, one-time costs such as model loading and initialization are excluded for all LLM-based methods, including EA4MP and Cerebro. We divide the runtime into two parts: feature extraction and classification reasoning. Feature extraction refers to the process of obtaining the required features from a package, while classification reasoning refers to the forward inference process after the features have been obtained. The right side of Fig. 8 shows the average runtime of each baseline method. We found that the rule-based methods OSS Detect Backdoor and Bandit4Mal, and the machine learning-based methods Amalfi, SAP, and MPHunter, had the best detection efficiency, with an average runtime of less than 1 second. However, their performance was worse than the MVRFDet method and the LLM-based methods. The LLM-based methods EA4MP and Cerebro had an average runtime of approximately 10 s. Specifically, the total runtime of EA4MP was 8.3 s, consisting of 0.07 s for feature extraction and 8.2 s for classification reasoning, while the total runtime of Cerebro was 13.4 s, consisting of 0.08 s for feature extraction and 13.3 s for classification reasoning. Although these methods have relatively excellent detection performance, they are insufficient for the high-efficiency requirements of large-scale PyPI package detection. Compared to rule-based and machine learning-based methods, the MVRFDet method has a higher average runtime, but achieves better detection performance. The VirusTotal online platform has the worst detection efficiency, mainly due to its integration of multiple antivirus engines, security tools, and dynamic sandbox tools, resulting in longer processing times for initial package analysis. In summary, the MVRFDet method achieves a better trade-off between runtime overhead and detection performance compared to baseline methods.

Table 4
Performance comparison results of different variant approachs on graph encoder.

Approach	Precision	Recall	F1-score
API CG-only	89.8	85.1	87.4
CFG-only	92.4	88.3	90.3
API CG + CFG w/o SFF	95.6	94.7	95.1
GE w/ GCN	96.2	95.3	95.7
GE w/ GGNN	94.8	93.9	94.3
MVRFDet	**98.4**	**96.8**	**97.6**

Table 5
Performance comparison results of different numbers of convolutional layers on CNN.

Conv layer	Precision	Recall	F1-score
1-conv	89.3	82.1	85.5
2-conv	93.6	91.7	92.6
3-conv	**98.4**	**96.8**	**97.6**
4-conv	96.5	93.2	94.8
5-conv	92.1	90.3	91.2

4.5. Hyperparameter sensitivity analysis (RQ4)

Sensitivity of the number of convolution layers. The MVRFDet method uses CNN to extract three view features from packages for malicious package detection. The number of convolutional layers in a CNN is an important parameter, directly affecting the detection performance of the MVRFDet method. To investigate the impact of different numbers of convolutional layers on the performance of MVRFDet, we conducted parameter sensitivity analysis experiments as follows.

Setup. The x-conv represents the number of convolutional layers used in the CNN model in MVRFDet.

Results and discussion. As shown in Table 5, we tested the performance of the MVRFDet method with different numbers of convolutional layers in the CNN model. Experimental results show that configuring 3 convolutional layers in the CNN exhibits the best performance. Overall, the model's performance first increases and then decreases with the increase of the number of convolutional layers, reaching a peak at 3 layers. We attribute this phenomenon to the dual effect of network depth. On the one hand, too few convolutional layers limit the representation learning ability of the CNN model, making it unable to capture complex features. On the other hand, an excessively deep network triggers a performance degradation. This is consistent with the conclusions of the ResNet [52] study, that is, in image classification tasks, excessive network depth leads to accuracy saturation and a sharp decline. Given the relatively limited size of the dataset used in this study, the degradation effect appears earlier than in large-scale image datasets. Therefore, setting the number of convolutional layers to 3 is a reasonable choice.

Sensitivity of incomplete sensitive API lists. Since the sensitive API list is used to localize suspicious API code context snippets, its completeness may affect downstream code and graph feature extraction. To evaluate the robustness of MVRFDet to an incomplete sensitive API list, we conduct a sensitivity analysis by deliberately removing APIs from the generated list and observing the resulting performance changes.

Setup. We consider two removal strategies. In the random removal setting, 10%–50% of APIs are randomly removed from the sensitive API list. To reduce randomness, each setting is repeated five times and the average result is reported. In the high-centrality removal setting, the top-ranked APIs according to their centrality values are removed,

Table 6
Sensitivity analysis of MVRFDet under incomplete sensitive API lists.

Setting	Removal ratio	Precision	Recall	F1-score
Full list	0%	**98.4**	**96.8**	**97.6**
Random	10%	98.1	96.2	97.1
Random	20%	97.7	95.4	96.5
Random	30%	97.1	94.3	95.7
Random	40%	96.3	92.8	94.5
Random	50%	95.2	91.1	93.1
High-centrality	10%	97.2	94.7	95.9
High-centrality	20%	95.9	92.5	94.2
High-centrality	30%	94.6	89.8	92.1

simulating a more challenging case where highly influential sensitive APIs are missed.

Results and discussion. Table 6 shows the experimental results. Overall, MVRFDet degrades gradually rather than abruptly as the sensitive API list becomes incomplete. Under random removal, when 10%, 20%, and 30% of APIs are removed, the F1-score decreases from 97.6% to 97.1%, 96.5%, and 95.7%, respectively. Even when 50% of APIs are removed, MVRFDet still achieves an F1-score of 93.1%. This indicates that the model is not entirely dependent on a perfectly complete sensitive API list. The metadata view and the global API call graph view can still provide complementary evidence when some suspicious API contexts are missed.

The performance drop is more significant under high-centrality removal. When the top 10%, 20%, and 30% high-centrality APIs are removed, the F1-score decreases to 95.9%, 94.2%, and 92.1%, respectively. The recall decreases more sharply than precision, indicating that missing critical APIs mainly leads to more false negatives. This result confirms that high-centrality APIs play an important role in locating malicious behaviors and also validates the necessity of using centrality analysis before LLM-based verification. These findings suggest that MVRFDet is reasonably robust to partial incompleteness of the sensitive API list, but dynamically maintaining and updating the list remains important for detecting newly evolving malicious behaviors.

5. Discussions and limitations

Although our MVRFDet has achieved cutting-edge results, it is imperative to critically analyze its shortcomings and potential hurdles in practical applications to ensure a comprehensive evaluation.

Potential threat from offline sensitive API knowledge construction. The sensitive API list in MVRFDet is constructed offline using verified malicious packages and serves as a reusable security knowledge base for suspicious API localization. This design reflects practical security deployment scenarios, where threat intelligence knowledge is continuously accumulated and maintained by security analysts. However, from a strict machine learning evaluation perspective, constructing such knowledge using all available malicious samples may introduce potential dependency between historical samples and feature extraction. Although the sensitive API list does not contain package-specific information or participate in classifier optimization, its influence on suspicious code localization should be carefully considered. Future work will investigate leakage-free knowledge updating strategies, where sensitive API knowledge is incrementally constructed only from available historical training data or continuously verified threat intelligence streams.

Challenges posed by data imbalance in real-world environments. In this study, we utilized datasets comprising an equal distribution of malicious and benign packages sourced from various repositories. This approach was designed to ensure a fair assessment of the model's discriminative capabilities during its development. However, we recognize that the actual PyPI environment is characterized by a dominance of benign packages, resulting in a severe class skew. Under such real-world imbalance, the reported precision and recall may shift from the balanced evaluation setting. In particular, even a small false positive rate may lead to a considerable number of benign packages being incorrectly flagged, thereby reducing practical precision. Meanwhile, recall depends on the decision threshold: a conservative threshold can reduce false positives but may miss stealthy malicious packages, whereas an aggressive threshold can improve recall at the cost of more false positives. Therefore, when deploying MVRFDet in real-world package registries, threshold tuning and probability calibration can be adopted to explicitly control the trade-off between false positives and false negatives. Another practical strategy is to use MVRFDet as a first stage high risk package filter, followed by manual inspection or dynamic analysis for packages with high malicious probabilities.

Resilience against code obfuscation. Malicious actors often resort to code obfuscation to bypass static analysis tools. Given the open-source transparency of the PyPI ecosystem, extensive obfuscation increases the likelihood of detection. Consequently, attackers typically limit obfuscation to parameters within a few critical functions. As MVRFDet adopts multi-view representation fusion of metadata, graph structure, and code semantics, it remains effective at identifying these common obfuscation tactics. Nevertheless, the model struggles against more sophisticated techniques, such as comprehensive package obfuscation or the execution of malicious code through entirely dynamic means. In summary, while MVRFDet offers a degree of resistance to obfuscation, addressing complex variants remains an open challenge that warrants further investigation.

Potential evasion strategies against multi-view detection. Although multi-view representation fusion improves the robustness of MVRFDet against isolated feature manipulation, an adaptive attacker may deliberately target specific views after understanding the detection pipeline. It is therefore necessary to discuss the potential evasion surface of MVRFDet and clarify the remaining limitations of the current design.

Evasion against the metadata view. The metadata view relies on seven fields extracted from the PKG-INFO file, including package name, summary, homepage, author, author Email, license, and description. Attackers may imitate legitimate packages by completing missing fields, copying the writing style of benign package descriptions, or providing seemingly plausible project links and maintainer information. Such manipulation may weaken metadata-based anomaly signals and reduce the effectiveness of metadata-only detection. However, metadata is used as a complementary channel in MVRFDet rather than as the sole classification basis. Consequently, metadata camouflage alone is unlikely to fully bypass the detector when suspicious code semantics or graph structures remain observable. A stronger defense should further incorporate metadata consistency checks, such as the temporal evolution of package versions, maintainer histories, repository provenance, and cross-field consistency.

Evasion against the graph view. The graph view combines a package-level API call graph with local control flow graphs constructed from suspicious API code context snippets. An attacker may attempt to weaken graph-based signals by distributing malicious logic across multiple functions or files, inserting unrelated benign operations, restructuring control flow, or relying on indirect and runtime-resolved invocations. These strategies may dilute the structural salience of sensitive API nodes, perturb graph topology, or prevent static analysis from recovering complete call relationships. In particular, malicious behaviors that are assembled or resolved only at runtime remain difficult to represent accurately through static graphs. Future work should investigate graph denoising techniques, interprocedural analysis, and selective lightweight dynamic analysis for packages with incomplete or anomalous static graphs.

Coordinated cross-view evasion. A more capable attacker may simultaneously craft plausible metadata and restructure malicious code to weaken both metadata and graph signals. The advantage of MVRFDet is that its heterogeneous feature channels impose a higher evasion cost: an attacker must consistently manipulate package identity information, graph topology, and suspicious code semantics. Nevertheless, multi-view fusion does not provide a formal guarantee against fully adaptive adversaries. The current results should therefore be interpreted as

evidence of improved robustness under ordinary malicious package distributions rather than complete resistance to adversarial manipulation. In future work, we plan to evaluate MVRFDet under adversarial environments and explore provenance-aware metadata modeling, robust graph representation learning, and hybrid static-dynamic analysis.

Adaptability across different ecosystems. The current implementation of MVRFDet is specifically tailored for Python packages within the PyPI ecosystem. Although the underlying concepts of multi-view representation fusion and offline automated sensitive API list extraction are theoretically universal, their efficacy in other open-source registries like NPM or Maven remains to be validated. Distinct ecosystems often exhibit significant variations in code architecture, metadata standards, package management protocols, and attack vectors. As a result, a direct port of MVRFDet to other platforms may yield suboptimal performance. In the future, we plan to explore domain-adversarial training for learning ecosystem-invariant graph and code representations, as well as ecosystem-specific adaptation of the sensitive API extraction process through prompt tuning and centrality analysis-based candidate selection. Lightweight ecosystem-specific adapters may also be incorporated into the multi-view fusion framework while keeping the main encoders shared. The long-term objective is to establish a unified security framework for the software supply chain that spans multiple ecosystems.

6. Conclusion and future directions

In this paper, we propose a novel multi-view representation fusion detection method called MVRFDet, to address the increasingly serious malicious package threat in the PyPI ecosystem. MVRFDet constructs a multi-level and comprehensive detection model by fusing metadata, graph structure, and code semantic representation of the package. To reduce runtime overhead and reliance on expert experience, we propose an offline automated sensitive API list extraction method combining centrality analysis and LLM. To adaptively fuse the three views of representation, we design a channel attention module, effectively improving the model's malicious package detection capability. Extensive experimental evaluations validate the effectiveness of MVRFDet. Furthermore, efficiency evaluation results demonstrate that the method achieves a good trade-off between runtime overhead and detection performance.

In the future, we aim to explore feature extraction and fusion mechanisms across different open-source ecosystems. Although this paper utilizes offline sensitive API list extraction to alleviate runtime overhead to some extent, maintaining high-precision detection under low latency requirements remains a challenge when facing massive and frequently released packages in the repositories. Future plans include researching a more lightweight model architecture to achieve truly real-time large-scale monitoring. As adversaries' obfuscation techniques continue to evolve, future efforts will focus on exploring ways to improve the model's generalization ability and robustness against unknown attack patterns and highly obfuscated code.

CRediT authorship contribution statement

Jiale Yan: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation; **Bo Zhao:** Writing – review & editing, Supervision, Resources, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research is funded by the National Natural Science Foundation of China (62572356). The authors would like to thank the reviewers for their insightful comments and useful suggestions.

Data availability

Data will be made available on request.

References

[1] K. Gao, R. He, B. Xie, M. Zhou, Characterizing deep learning package supply chains in PyPI: domains, clusters, and disengagement, ACM Trans. Softw. Eng. Methodol. 33 (4) (2024) 97:1–97:27. https://doi.org/10.1145/3640336
[2] Y. Gu, L. Ying, Y. Pu, X. Hu, H. Chai, R. Wang, X. Gao, H. Duan, Investigating package related security threats in software registries, in: 44th IEEE Symposium on Security and Privacy, San Francisco, CA, USA, IEEE, 2023, pp. 1578–1595. https://doi.org/10.1109/SP46215.2023.10179332
[3] P. Ladisa, H. Plate, M. Martinez, O. Barais, SoK: taxonomy of attacks on open-source software supply chains, in: 44th IEEE Symposium on Security and Privacy, San Francisco, CA, USA, IEEE, 2023, pp. 1509–1526. https://doi.org/10.1109/SP46215.2023.10179304
[4] X. Gao, X. Sun, S. Cao, K. Huang, D. Wu, X. Liu, X. Lin, Y. Xiang, MalGuard: towards real-time, accurate, and actionable detection of malicious packages in PyPI ecosystem, in: 34th USENIX Security Symposium, Seattle, WA, USA, USENIX Association, 2025, pp. 4741–4758.
[5] VirusTotal, VirusTotal: analyse suspicious files, domains, ips and urls to detect malware and other breaches, automatically share them with the security community, 2025, https://www.virustotal.com/gui/home/upload.
[6] Microsoft, OSS detect backdoor: identifies potential backdoors and malicious code within a package, 2022, (https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor).
[7] D.-L. Vu, A fork of bandit tool with patterns to identifying malicious python code, 2020, https://github.com/lyvd/bandit4mal.
[8] S. Halder, M. Bewong, A. Mahboubi, Y. Jiang, M.R. Islam, M.Z. Islam, R.H.L. Ip, M.E. Ahmed, G.S. Ramachandran, M.A. Babar, Malicious package detection using metadata information, in: Proceedings of the ACM on Web Conference 2024, Singapore, ACM, 2024, pp. 1779–1789. https://doi.org/10.1145/3589334.3645543
[9] A. Sejfia, M. Schäfer, Practical automated detection of malicious npm packages, in: 44th IEEE/ACM 44Th International Conference on Software Engineering, Pittsburgh, PA, USA, ACM, 2022, pp. 1681–1692.
[10] P. Ladisa, S.E. Ponta, N. Ronzoni, M. Martinez, O. Barais, On the feasibility of cross-language detection of malicious packages in npm and PyPI, in: Annual Computer Security Applications Conference, Austin, TX, USA, ACM, 2023, pp. 71–82. https://doi.org/10.1145/3627106.3627138
[11] W. Liang, X. Ling, J. Wu, T. Luo, Y. Wu, A needle is an outlier in a haystack: hunting malicious PyPI packages with code clustering, in: 38th IEEE/ACM International Conference on Automated Software Engineering, Luxembourg, IEEE, 2023, pp. 307–318. https://doi.org/10.1109/ASE56229.2023.00085
[12] X. Sun, X. Gao, S. Cao, L. Bo, X. Wu, K. Huang, $1+1>2$: integrating deep code behaviors with metadata features for malicious PyPI package detection, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, ACM, 2024, pp. 1159–1170. https://doi.org/10.1145/3691620.3695493
[13] J. Zhang, K. Huang, Y. Huang, B. Chen, R. Wang, C. Wang, X. Peng, Killing two birds with one stone: malicious package detection in NPM and PyPI using a single model of malicious behavior sequence, ACM Trans. Softw. Eng. Methodol. 34 (4) (2025) 1–28.
[14] N. Zahan, P. Burckhardt, M. Lysenko, F. Aboukhadijeh, L.A. Williams, Leveraging large language models to detect NPM malicious packages, in: 47th IEEE/ACM International Conference on Software Engineering, Ottawa, ON, Canada, IEEE, 2025, pp. 2625–2637. https://doi.org/10.1109/ICSE55347.2025.00146
[15] X. Zheng, C. Wei, S. Wang, Y. Zhao, P. Gao, Y. Zhang, K. Wang, H. Wang, Towards robust detection of open source software supply chain poisoning attacks in industry environments, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, ACM, 2024, pp. 1990–2001. https://doi.org/10.1145/3691620.3695262
[16] C. Huang, N. Wang, Z. Wang, S. Sun, L. Li, J. Chen, Q. Zhao, J. Han, Z. Yang, L. Shi, DONAPI: malicious NPM packages detector using behavior sequence knowledge mapping, in: 33rd USENIX Security Symposium, 2024, pp. 3765–3782.
[17] D. Vu, Z. Newman, J.S. Meyers, Bad snakes: understanding and improving python package index malware scanning, in: 45th IEEE/ACM International Conference on Software Engineering, Melbourne, Australia, IEEE, 2023, pp. 499–511. https://doi.org/10.1109/ICSE48619.2023.00052
[18] J. Wang, Z. Li, J. Qu, D. Zou, S. Xu, Z. Xu, Z. Wang, H. Jin, MalPacDetector: an LLM-based malicious NPM package detector, IEEE Trans. Inf. Forensics Secur. 20 (2025) 6279–6291. https://doi.org/10.1109/TIFS.2025.3580336
[19] S.R. Addula, S. Norozpour, M. Amin, Risk assessment for identifying threats, vulnerabilities and countermeasures in cloud computing, Jordanian J. Inform. Comput. 2025 (1) (2025) 37–48.
[20] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, Y. Liu, An empirical study of malicious code in pyPI ecosystem, in: 38th IEEE/ACM International Conference on Automated Software Engineering, Luxembourg, IEEE, 2023, pp. 166–177. https://doi.org/10.1109/ASE56229.2023.00135
[21] N. Zahan, P. Burckhardt, M. Lysenko, F. Aboukhadijeh, L.A. Williams, MalwareBench: malware samples are not enough, in: 21st IEEE/ACM International Conference on Mining Software Repositories, Lisbon, Portugal, ACM, 2024, pp. 728–732. https://doi.org/10.1145/3643991.3644883
[22] PyPI, PyPI official statistics, 2025, (https://pypi.org/stats).

[23] D. Gonzalez, T. Zimmermann, P. Godefroid, M. Schaefer, Anomalicious: automated detection of anomalous and potentially malicious commits on GitHub, in: 43rd IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, Madrid, Spain, IEEE, 2021, pp. 258–267. https://doi.org/10.1109/ICSE-SEIP52600.2021.00035

[24] R. Goyal, G. Ferreira, C. Kästner, J.D. Herbsleb, Identifying unusual commits on GitHub, J. Softw. Evol. Process. 30 (1) (2018). https://doi.org/10.1002/SMR.1893

[25] A. Cao, B. Dolan-Gavitt, What the fork? Finding and analyzing malware in GitHub forks, in: Proceedings 2022 NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb), San Diego, CA, USA, 2022. https://doi.org/10.14722/madweb.2022.23001

[26] Y. Gu, L. Ying, H. Chai, C. Qiao, H. Duan, X. Gao, Continuous intrusion: characterizing the security of continuous integration services, in: 44th IEEE Symposium on Security and Privacy, San Francisco, CA, USA, IEEE, 2023, pp. 1561–1577. https://doi.org/10.1109/SP46215.2023.10179471

[27] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, A. Machiry, Characterizing the security of github CI workflows, in: 31st USENIX Security Symposium, Boston, MA, USA, USENIX Association, 2022, pp. 2747–2763.

[28] C. Lamb, S. Zacchiroli, Reproducible builds: increasing the integrity of software supply chains, IEEE Softw. 39 (2) (2022) 62–70. https://doi.org/10.1109/MS.2021.3073045

[29] C. Li, Y. Wu, W. Shen, Z. Zhao, R. Chang, C. Liu, Y. Liu, K. Ren, Demystifying compiler unstable feature usage and impacts in the rust ecosystem, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, Lisbon, Portugal, ACM, 2024, pp. 24:1–24:13. https://doi.org/10.1145/3597503.3623352

[30] M. Xu, Y. Wang, S. Cheung, H. Yu, Z. Zhu, Insight: exploring cross-Ecosystem vulnerability impacts, in: 37th IEEE/ACM International Conference on Automated Software Engineering, Rochester, MI, USA, ACM, 2022, pp. 58:1–58:13. https://doi.org/10.1145/3551349.3556921

[31] K. Huang, B. Chen, C. Xu, Y. Wang, B. Shi, X. Peng, Y. Wu, Y. Liu, Characterizing usages, updates and risks of third-party libraries in java projects, Empir. Softw. Eng. 27 (4) (2022) 90. https://doi.org/10.1007/S10664-022-10131-8

[32] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, X. Peng, Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem, in: 44th IEEE/ACM 44th International Conference on Software Engineering, Pittsburgh, PA, USA, ACM, 2022, pp. 672–684. https://doi.org/10.1145/3510003.3510142

[33] M. Zimmermann, C. Staicu, C. Tenny, M. Pradel, Small world with high risks: a study of security threats in the npm ecosystem, in: 28th USENIX Security Symposium, Santa Clara, CA, USA, USENIX Association, 2019, pp. 995–1010.

[34] W. Enck, L.A. Williams, Top five challenges in software supply chain security: observations from 30 industry and government organizations, IEEE Secur. Priv. 20 (2) (2022) 96–100. https://doi.org/10.1109/MSEC.2022.3142338

[35] S. Neupane, G. Holmes, E. Wyss, D. Davidson, L. De Carli, Beyond typosquatting: an in-depth look at package confusion, in: 32nd USENIX Security Symposium, Anaheim, CA, USA, USENIX Association, 2023, pp. 3439–3456.

[36] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, Association for Computational Linguistics, 2019, pp. 4171–4186. https://doi.org/10.18653/V1/N19-1423

[37] N. Reimers, I. Gurevych, Sentence-BERT: sentence embeddings using siamese BERT-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, Hong Kong, China, Association for Computational Linguistics, 2019, pp. 3980–3990. https://doi.org/10.18653/V1/D19-1410

[38] L. Li, J. Wang, H. Quan, Scalpel: the python static analysis framework, arXiv:2202.11840 (2022).

[39] Z. Yu, M. Wen, X. Guo, H. Jin, Maltracker: a fine-grained NPM malware tracker copiloted by LLM-enhanced dataset, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, Vienna, Austria, ACM, 2024, pp. 1759–1771. https://doi.org/10.1145/3650212.3680397

[40] N. Li, S. Wang, M. Feng, K. Wang, M. Wang, H. Wang, MalWuKong: towards fast, accurate, and multilingual detection of malicious code poisoning in OSS supply chains, in: 38th IEEE/ACM International Conference on Automated Software Engineering, Luxembourg, IEEE, 2023, pp. 1993–2005. https://doi.org/10.1109/ASE56229.2023.00073

[41] Y. Wu, W. Suo, S. Feng, D. Zou, W. Yang, Y. Liu, H. Jin, MalScan: android malware detection based on social-network centrality analysis, IEEE Trans. Dependable Secur. Comput. 22 (5) (2025) 5464–5479. https://doi.org/10.1109/TDSC.2025.3561944

[42] A.A. Hagberg, D.A. Schult, P.J. Swart, Exploring network structure, dynamics, and function using NetworkX, in: Proceedings of the Python in Science Conference, SciPy, 2008, pp. 11–15.

[43] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al., DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning, Nature 645 (8081) (2025) 633–638.

[44] OpenAI, Introducing GPT-5, 2025, (https://openai.com/index/introducing-gpt-5/).

[45] D. Guo, C. Xu, N. Duan, J. Yin, J.J. McAuley, LongCoder: a long-range pre-trained language model for code completion, in: International Conference on Machine Learning, Honolulu, Hawaii, USA, 202, PMLR, 2023, pp. 12098–12107.

[46] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, Q. Liu, TFE-GNN: a temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification, in: Proceedings of the ACM Web Conference 2023, Austin, TX, USA, ACM, 2023, pp. 2066–2075.

[47] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: 6th International Conference on Learning Representations, Vancouver, BC, Canada, OpenReview.net, 2018.

[48] W. Cong, M. Ramezani, M. Mahdavi, On provable benefits of depth in training graph convolutional networks, in: Advances in Neural Information Processing Systems, 34, Curran Associates, Inc., 2021, pp. 9936–9949.

[49] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: a robustly optimized BERT pretraining approach, arXiv:1907.11692 (2019).

[50] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: 5th International Conference on Learning Representations, Toulon, France, OpenReview.net, 2017.

[51] Y. Li, D. Tarlow, M. Brockschmidt, R.S. Zemel, Gated Graph Sequence Neural Networks, in: 4th International Conference on Learning Representations, San Juan, Puerto Rico, 2016. arXiv:1511.05493.

[52] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, IEEE Computer Society, 2016, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90